

Reverse-Engineering an ASIC

From mask polygons to the string the chip prints

Vladislav Shapovalov

figurezig@protonmail.ch

August 2026

Abstract—A finished chip is shipped as polygons on mask layers. Every name its designer wrote is gone, because the manufacturing format has nowhere to keep them. Given one such file and no other information, this write-up recovers what the chip computes.

The route has two halves. Geometry becomes a netlist by tracing which shapes are electrically connected, a step validated beforehand on a smaller design whose answer is published. The netlist then becomes a design by expanding each flip-flop’s next-state function and reading the registers out of the resulting algebra; where the algebra stopped being readable, the circuit was measured instead, one input bit at a time. The recovered logic — two counters, a shift register, four accumulators and a single-cycle verdict — was written back out as behavioural Verilog and proven equivalent to the gates it came from.

The chip turns out to check a Star Battle puzzle on an 11×11 grid. The puzzle has one solution, and feeding it in makes the chip print (** TWO STARS **). As an independent check, replaying the vendor’s own sample waveform through the extracted netlist reproduces their recorded output on all 312 samples.

Index Terms—hardware reverse engineering, GDSII, netlist extraction, equivalence checking

I. THE PROBLEM

The target is one file: `puzzle.gds`, a layout in the format a chip is sent to a factory in. It holds polygons on numbered mask layers and nothing else — no netlist, no schematic, no module hierarchy. It does carry text labels, and those name the ports: `clk`, `rst_n`, `enable`, a one-bit `I`, and the outputs `success` and `O[7:0]`. A recorded waveform of the original chip is supplied alongside it, driven with an input that does *not* make `success` go high.

Separately there is a warm-up: a much smaller design, published together with every intermediate file of the flow that built it — source, synthesized netlist, the same netlist with power connected, placed-and-routed database, and final layout. The warm-up is not part of the target and its answer is already public. Its role here is entirely different, and Section II-B explains what that role is.

The task is to find an input that makes `success` go high, and then read what the chip prints on `O`.

II. GETTING A NETLIST ONE CAN TRUST

A. Reading the file before writing any code

The first command I ran was the inspector, and it answered the question that mattered most (Listing 1).

The standard cell names survived, and that determines how hard the rest of the work is. Had they been stripped, I would have had to identify each cell from its geometry, or in the worst case recognise individual transistors as gate poly crossing

```
$ asicrev inspect puzzle.gds --cells
cells in library: 81
204 sky130_fd_sc_hd__decap_3
84 sky130_fd_sc_hd__dftrtp_2
49 sky130_fd_sc_hd__nor2_2
...
21 INTERNAL_3
15 INTERNAL_7
```

Listing 1: The first command run against the layout. Every name but the last two is a documented standard cell, which settles how hard the rest of the work is.

diffusion and fold the resulting series-parallel networks back into Boolean functions. With the names intact, cell behaviour is a table lookup and the only remaining problem is geometric: decide which shapes are electrically the same wire.

Two other things are visible in that listing. There are 84 flip-flops with an asynchronous reset, which sets a rough scale for the design. And two cells named `INTERNAL` belong to no standard library; they are not part of the circuit at all, and Appendix A explains what they are.

B. Validating the extractor before extending it

The obvious move is to start on the real design. I did not, because of one property of this problem that is easy to underestimate: **a faulty extractor produces a netlist that looks entirely reasonable**. The cell count is right, the net count is plausible, no pin is left dangling, and it simulates without complaint. It simply computes a different function, and nothing in the output indicates this.

The warm-up is the only place where that can be caught, because it is the same flow with a published answer at four levels of detail. What follows is the order in which I actually used it, because the order is the point: each artefact tests a strictly narrower hypothesis than the last, so a failure localises itself.

First, does the reader read? Running the inspector on the warm-up layout and counting the placements per cell type gives a histogram. The synthesized netlist gives the same histogram directly. They agreed exactly — 79 logic cells, 16 flip-flops, 16 multiplexers, and so on. That says the record parser, the hierarchy flattening and the transform composition are right, and it says nothing whatever about connectivity.

Second, is the connectivity right at all? The routed database lists, for every net, which instance pins it touches. Comparing that against the extraction is one net at a time rather than all-or-nothing, so it localises errors. This is where the abutment-box

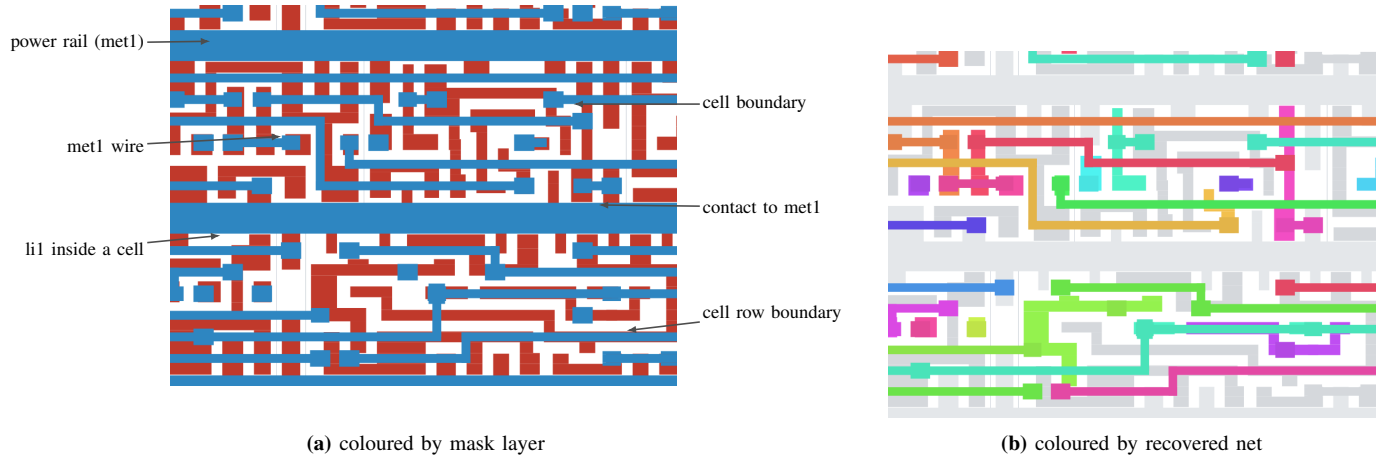


Fig. 1: An $8 \times 6 \mu\text{m}$ window of the layout, showing only the two lowest conducting levels so that the cells are visible. On the left, colour is the mask shape sits on: local interconnect in red, first metal in blue. The wide horizontal blue bands are the power rails that every cell straddles, and the faint vertical lines are cell boundaries. On the right, the same shapes coloured by the electrical node the extraction assigned them, with the supplies greyed out. Turning the left picture into the right one is the whole extraction problem.

convention surfaced (§II-D), and once the comparison itself was correct it scored 230/230 placements and 84/84 nets.

Third, is it the same circuit? Pin sets can match while the wiring is still wrong, so the next artefact is the netlist itself, compared as a graph isomorphism. The first two attempts failed, and both failures were in my checker rather than in the extraction: an edge comparison that assumed two incomparable orderings, and a colour refinement run separately on the two graphs (§II-D). With those fixed, the comparison passes with no search at all.

Fourth, does something that shares none of my assumptions agree? My isomorphism check and my extractor were written by the same person in the same afternoon. An external SAT-based equivalence checker is not. It needs the two designs to expose matching internal node names before it can pair anything, so the isomorphism from the previous step was used to rename the recovered netlist, after which 380 of 380 nodes were proven equivalent.

Fifth, does the thing compute what the documentation says? The warm-up’s published description says its output goes high when two serially loaded eight bit values sum to 496. Simulating the recovered netlist over all 2^{16} input pairs gives exactly 15 that drive the output high, and all 15 satisfy $A + B = 496$.

Only after the fifth of these did I open `puzzle.gds`. The sequence is worth stating plainly because it is the whole methodological content of this report: five artefacts, five progressively narrower questions, and no work on the unknown design until each had been answered.

C. Two steps that break extraction

The algorithm is five lines. Flatten the placement hierarchy into absolute coordinates; cut every polygon into disjoint rectangles; merge rectangles that touch on the same conducting level, and merge across levels wherever a via joins them; the connected components are the nets; bind each cell’s pin labels

to the net underneath and print Verilog. Nearly all of that is bookkeeping. Two steps are not.

a) Bounding boxes are not good enough: Overlap tests are cheap between rectangles and awkward between polygons, so the tempting shortcut is to replace each polygon by its bounding box. Figure 2 shows why that fails, using a real 68-vertex piece of local interconnect taken from a clock buffer: its bounding box covers roughly twice the area the metal occupies, and any wire crossing the empty part is merged into it. The warm-up alone contains 108 non-rectangular conductor polygons, so this is a common case rather than a pathological one.

Lemma 1 (Exact rectangular decomposition). *Let P be a simple rectilinear polygon and let $y_1 < \dots < y_m$ be the distinct ordinates of its vertices. Fix a slab $S_i = (y_i, y_{i+1})$ and let E_i be the vertical edges of P spanning it, with abscissae sorted $x_{i,1} \leq \dots \leq x_{i,n_i}$. Then n_i is even and*

$$\text{int}(P) \cap (\mathbb{R} \times S_i) = \bigcup_{j=1}^{n_i/2} (x_{i,2j-1}, x_{i,2j}) \times S_i.$$

Proof. No vertex of P has an ordinate strictly inside S_i , so every edge meeting the open slab is vertical and spans it; the crossings of a horizontal line at height $y \in S_i$ are therefore exactly E_i , independent of y . A horizontal line meets a closed curve an even number of times, so n_i is even. By the Jordan-curve parity criterion a point of that line is interior exactly when an odd number of crossings lie to its left, which is the stated union. ■

b) A via conducts on overlap, not on contact: Two pieces of metal on the same level that share an edge are one wire, because abutting metal conducts. Reusing that test for vias is the natural next step, and it is incorrect: a via at the end of one wire routinely abuts the boundary of a neighbouring track on the level above, and under a touching test those two independent nets merge. Vias require strictly positive overlap

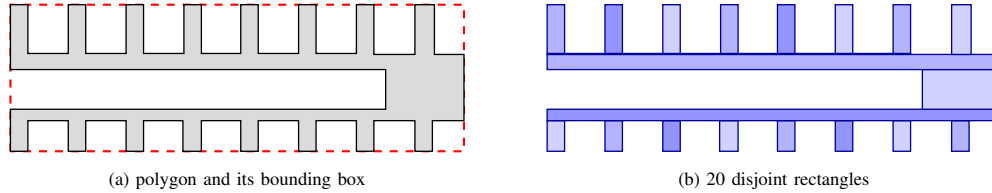


Fig. 2: A real 68-vertex conductor from `clkbuf_16`: (a) the shape and its bounding box, (b) the exact decomposition into 20 disjoint rectangles produced by Lemma 1.

area. The asymmetry between the two rules looks arbitrary until it causes a failure, and it is the kind of error that is only found by having a reference to check against.

D. Independent checks

Table I lists the five checks. The first of them nearly sent me the wrong way: it initially scored 96 of 230, which resembles a serious extraction fault. It was not one. A routed database records a cell's position as the lower-left corner of its abutment box *after* orientation, while a layout records the cell's own origin, and for the flipped rows that alternate down a standard-cell layout those differ by exactly the row height. Comparing transformed abutment boxes scores 230/230. The fault was in the checker.

Two remarks on the isomorphism, which does the real work here. Pin names must be folded into the edge labels, or the two inputs of an asymmetric gate become interchangeable. And the colour refinement has to run over *both* graphs against one shared signature table: refine them separately and identical structures receive different identifiers on each side, so the two colourings cannot be compared at all. With both of those right, the comparison finishes with no backtracking.

III. TAKING THE NETLIST APART

The same code on `puzzle.gds` reports 1618 placements, 130 137 polygons, 72 500 rectangles, 8 400 pin labels, 739 signal nets and 728 logic cells, in 60 ms. It also reports no unbound labels, no pin resolving to two nets, no unknown cells and no warnings; those four zeros are the extractor's own account of whether it understood the file.

That left 728 gates and 92 flip-flops, and no indication of what any of them do. Reading the netlist gate by gate is possible, but slow and error-prone, and I never did it. Instead, each flip-flop's next-state function was expanded symbolically through the combinational cloud until it was written purely in terms of other flip-flop outputs and the primary inputs.

That representation supports a specific kind of reasoning, and the rest of this section is that reasoning in the order it happened. Each step is the same shape: find a subset of the state that can be characterised without knowing anything about the rest, characterise it, and let the result decide which subset to look at next. Concretely — terms that appear in nearly every equation are global control, so they come first; registers whose equations never mention the input are autonomous, so they can be read in isolation; and the output's own equation, read backwards, says which registers the design considers important

without requiring any of them to be understood yet. Only what survives all three passes needs to be attacked by other means.

A. Following the enable

Almost every equation contains $\neg q_{36} \wedge \text{enable}$. That is a global enable; call it *step*. And q_{36} is interesting on its own,

$$q'_{36} = q_{36} \vee (\dots \wedge \text{step}), \quad q'_0 = q_{36} \vee q_0,$$

so q_{36} is sticky — once set it never clears — and q_0 is q_{36} delayed by one cycle, which makes $q_{36} \wedge \neg q_0$ true on exactly one cycle. Before understanding anything else about the design, I knew that the chip runs for some number of cycles and then makes a single decision.

B. The scan counters

Knowing there is a global enable and a finishing event, the natural next question is what governs *when* that event fires. Whatever it is cannot depend on the input, since the chip must finish regardless of what is fed to it, so the candidates are the flip-flops whose equations do not mention $\mathbb{I}$ at all. There are eight, and they read alike. Here is one, verbatim,

$$\begin{aligned} q'_{39} &= \neg((\neg q_{39} \wedge \neg q_{37} \wedge q_{38} \wedge q_{40}) \\ &\quad \vee \neg(q_{39} \oplus \text{step})) \\ &= \neg D \wedge (q_{39} \oplus \text{step}), \end{aligned}$$

where D abbreviates that four-variable term. A bit that toggles on every step is bit 0 of a counter, and D clears it. The neighbouring equations give the carry chain, $q'_{37} = q_{37} \oplus (q_{40} \wedge q_{39} \wedge \text{step})$ and so on, which pins the weights to $q_{39}=1$, $q_{40}=2$, $q_{37}=4$ and $q_{38}=8$. Substituting those into D shows that it decodes the value 10.

Four flip-flops are therefore a counter that runs from 0 to 10 and wraps, which is the first structural fact about the design and comes out of a single equation. Four more flip-flops have identical structure but are enabled by $\text{step} \wedge (\text{col}=10)$ — a second modulo-eleven counter, advancing when the first wraps — and q_{36} is set by $\text{step} \wedge (\text{col}=10) \wedge (\text{row}=10)$.

So the chip scans an 11×11 grid at one cell per clock and consumes exactly 121 input bits in row-major order. That also explains the sample waveform, which drives 242 bits: twice 121, with the second half ignored because `done` has latched.

TABLE I: Checks run against the warm-up design. Each answers a different question, and each can fail without the others noticing.

Check	Result	What it would have caught
Placement vs. the routed database	230/230 cells at the same position	mis-parsed placement transforms, or a wrong database unit
Connectivity vs. the same database	84/84 nets with identical pin sets	a missed or spurious connection, one net at a time
Graph isomorphism vs. the reference netlist	proven, with no search	any structural difference at all, since names cannot be compared
SAT equivalence, by an external tool	380/380 nodes proven	assumptions shared between my extractor and my own checker
Recovered function vs. the published description	high exactly when $A + B = 496$	everything, end to end

C. Reading the verdict backwards

At this point the design’s shape is known — it consumes 121 bits and then decides — but not what it decides. Rather than work forwards from the input, which means understanding the datapath, it is cheaper to work backwards from the output, which requires understanding nothing: whatever the verdict reads is by definition what the design considers important. Expanding q_2 , the flip-flop driving *success*, gives a large expression with an obvious shape,

$$q'_2 = (\neg q_0 \wedge q_{36}) ? (\neg q_{46} \wedge \neg q_{55} \wedge \Pi) : q_2,$$

where Π is a conjunction of about fifty terms, each of the form $\neg q_i \wedge q_j$ or $q_i \wedge \neg q_j$. Synthesis produces exactly that pattern from an equality test against a constant, since each term pins two bits. So at the finishing cycle two flags must be clear and three groups of registers must equal fixed constants.

Following each pair back to its own equation identifies those groups: eleven two-bit saturating counters, another eleven, and one eight-bit counter, required to equal 2, 2 and 22 respectively. Eleven counters of two marks each gives a total of twenty-two, which is consistent.

The flags q_{46} and q_{55} are both sticky vetoes. The simpler one,

$$q'_{55} = q_{55} \vee (\text{step} \wedge (\text{col}=10) \wedge (q_{54} ? (q_{56} \vee I) : \neg(q_{56} \wedge I))),$$

works out, over the four cases of (q_{54}, q_{56}) , to a two-bit count of the marks in the current row, cleared at each row end, with the condition “the count including this last cell is not 2”. Rows are therefore constrained as well, but enforced by a check-and-clear at the end of each row rather than by eleven parallel accumulators. That asymmetry is a choice a human made, and it survives into the silicon.

D. The shift register and its taps

Two things in the verdict are still unexplained: the second veto flag, and thirteen flip-flops that belong to no group identified so far. They turn out to be the same question. Those thirteen have trivial equations: $q'_{42} = I$, $q'_{45} = q_{47}$, $q'_{47} = q_{50}$, and so on. Following the chain from I gives a plain twelve-stage shift register. The remaining veto reads

$$q'_{46} = q_{46} \vee (I \wedge \text{step} \wedge ((\text{col} \neq 0) h_1 \vee (\text{col} \neq 10) h_{10} \vee h_{11} \vee (\text{col} \neq 0) h_{12})),$$

using taps 1, 10, 11 and 12 of that register. On a grid scanned row-major at eleven cells per row, a delay of 11 is the cell directly above, 10 is up-and-right, 12 is up-and-left, and 1 is the cell to the left — precisely the neighbours already seen when scanning in this order, with the column guards preventing a tap from wrapping past a row boundary. So marks may not touch, diagonals included, and the tap depth confirms the grid width a second time, independently of the counters.

E. What resisted reading

One group is left: the eleven counters the verdict compares but whose increment conditions have not been read. Everything so far came out of the equations. These did not. Their increment conditions expand into decodes of *row* and *col* hundreds of characters long, with no structure a human eye recovers, because synthesis had minimised eleven arbitrary membership functions into one shared network.

So I stopped reading and ran an experiment instead. The netlist compiles into straight-line Python, one expression per net in topological order, which makes a full 121-cycle run cost about a microsecond. Set exactly one input bit, run, record which counter moved, and repeat 121 times.

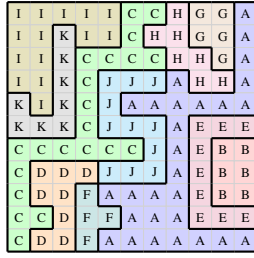
The first attempt reported twenty-seven groups, which is wrong: a perturbation also disturbs the output stage’s feedback register and the neighbourhood shift register. Restricting the observed set to the flip-flops in the support of the verdict — information the symbolic expansion had already provided — reduced it to exactly eleven. Plotted on the grid, those eleven groups are *connected regions* (Figure 3a). Nothing in the procedure asked for connectivity, so its appearance was the first real confirmation that the interpretation was correct.

IV. THE RECOVERED DESIGN

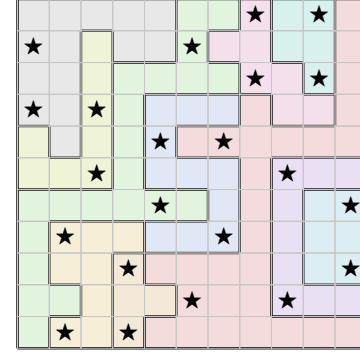
Collecting the equations gives the design in Figure 8, with its full next-state description in Figure 7 and every one of the 92 flip-flops accounted for in Table IV; all three are in Appendix B.

Read together they say: mark cells on an 11×11 grid, with exactly two marks in every row, in every column and in each of eleven irregular regions, and no two marks touching, diagonals included. That is a *Star Battle* puzzle.

Two features of the implementation are worth noting, because both are choices rather than consequences. The row constraint is enforced differently from the column and region constraints,



(a) recovered by perturbation



(b) the unique solution

Fig. 3: (a) The result of 121 experiments, each setting one input bit and recording which counter changed. Cells driving the same counter share a letter, and heavy borders mark where neighbours differ. The eleven groups come out connected, which is an output of the experiment rather than an input to it. (b) The one assignment the recovered constraints admit.

by a check-and-clear at the end of each row instead of eleven parallel accumulators. And the total mark count is checked against 22 even though the eleven column counters already imply it, which is a redundant test a synthesis tool would not have invented.

V. THE ANSWER

Backtracking over rows, placing two marks per row subject to the column, region and adjacency constraints, finds exactly **one** solution (Figure 3b) in under a second. Feeding its 121 bits in with `enable` held high drives `success` high on the cycle after the last bit.

The 23 flip-flops of block E are reached by no primary input, so rather than reverse their feedback sequence I characterised the block by simulation. It emits one fixed string, a character per clock, and then holds at zero. With the solution on the input, that string is

(* TWO STARS *)

which is the answer, and which names the rule the chip has just finished checking. The block stores three further strings for inputs that fail, and the way it chooses between them is worth a note of its own (Appendix A).

VI. INDEPENDENT CONFIRMATION

At this point the interpretation rests on my own reading of my own extraction. Two checks do not.

a) Against the manufacturer's own simulation: The sample waveform records 312 clock edges of the original chip's outputs. Replaying that exact stimulus through the extracted netlist reproduces them on **all 312 samples**, bit for bit, including the eight-bit bus, which spells TRY AGAIN because the sample input is deliberately wrong. That waveform was produced by the original design and was never an input to extraction.

Reaching it required two corrections, both of them sampling conventions rather than logic. The stimulus visible at a clock edge is the value from *before* that timestamp's updates, and a combinational output must be re-evaluated from the post-edge state rather than read from the values computed during the

edge. Before those corrections the replay matched 292 of 312. A near-match of that kind is more dangerous than an outright failure, because it is indistinguishable from a subtle extraction fault.

b) Against the reconstruction: The equations of Figure 7 were written out as 261 lines of behavioural Verilog and checked against the gate netlist they came from. They agree on `success` and on both internal veto flags for every cycle of the solution, the two degenerate grids and 200 pseudo-random grids. A miter between the two designs then yields no SAT counterexample to a depth of 128 cycles, a problem of 1 209 641 variables and 3 207 542 clauses that took 332 minutes to discharge.

That bound is not merely deep, it is complete. The design is a straight-line scan of 121 cells which latches `done` on the last one and thereafter ignores its input, so every behaviour it has is reached within 121 cycles of reset. The reconstruction and the extracted gates are equivalent on `success`, not merely in agreement on the inputs that were tried.

VII. ERRORS MADE ALONG THE WAY

Each item below cost real time, and most share the same character: the code was incorrect in a way that still produced plausible output.

a) A guard that discarded a whole class of shape: The decomposer began with `if (ring.size() < 4) return {};`, which is reasonable, since a closed rectilinear ring needs four vertices. But a triangle has three, and the correct response to a triangle is not “nothing” but “not rectilinear, fall back to the bounding box and report it”. This was caught by a unit test written before the extractor ever saw real data.

b) Comparing two sorted lists that are not comparable: The isomorphism checker walked both candidates' edge lists in parallel. Edges are sorted by pin name and neighbour index, and neighbour indices are internal to each graph, so the two orders have nothing to do with each other. The check reported a difference between a netlist and an isomorphic copy of itself.

c) A parser that consumed the section it was about to read: The routed database ends each section with `END <NAME>` and

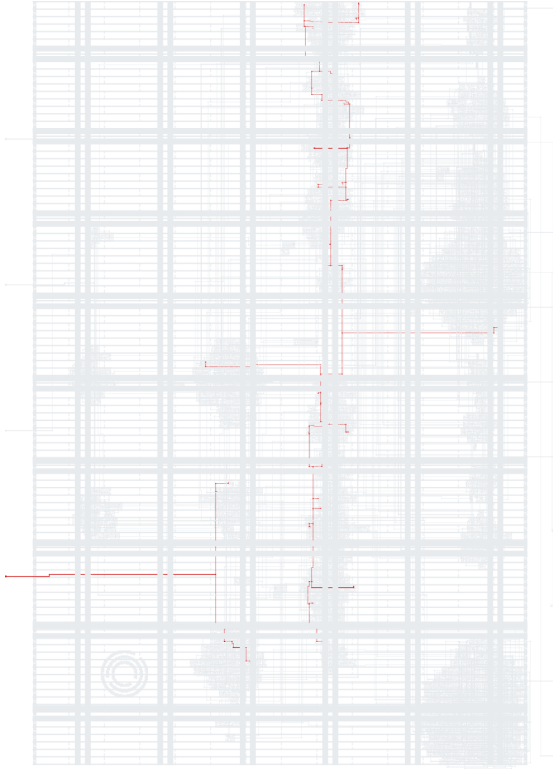


Fig. 4: One recovered net drawn on the die: the serial input in red, everything else in grey. It enters at the left, crosses on an upper metal level and drops into flip-flops in several rows. The grey is 72 500 rectangles; the red is one connected component of the union-find. This is the cheapest sanity check available, since a net that looks like scattered confetti rather than a tree means the merge is wrong.

no semicolon. The reader finished a section by skipping to the next semicolon, which lay several tokens into the *following* header. The connectivity check then reported “0 of 0 nets matched” and looked like a success.

d) The harness was wrong, not the extractor: See the 96/230 placement score above. A verification harness is code, and deserves the same suspicion as the thing it checks.

e) A library cell whose model disagrees with the flow: The well-tap cell is documented with four terminals and instantiated by the flow with two, so the power-aware comparison failed on exactly one class of node. When a vendor model and a vendor flow disagree, the flow is the authority for what a netlist will look like.

f) One that was not a fault at all: A reconstruction already shown correct by cross-probing suddenly gave the wrong verdict in a second testbench. The cause was that testbench’s clock: it drove `clk` low and then high inside the loop body, so the first falling edge was a no-op and every later sample was taken on the wrong side of the edge. Forty minutes went into re-deriving equations that were already right.

VIII. NOTES

a) Hidden in the layout: Three things in these files are jokes rather than circuitry: a message in Morse code placed outside the die, a set of four different output strings where one

would do, and a timestamp that existed for exactly one second in history. Appendix A gives each one together with the step that turned it up.

b) The software: One C++20 program of 7.1 kLOC with no EDA dependencies: stream reader and hierarchy flattening, geometry, technology and cell library, netlist IR with Verilog input and output and the isomorphism check, simulation, SVG rendering, and a driver with five subcommands. On top of it sits 0.6 kLOC of Python: a compiler from netlist to straight-line code, the solver, and the RTL recovery. There are forty-two tests, of which the useful ones are property-shaped — each cell’s Boolean function is re-derived as a truth table and compared against the process kit’s own model, the decomposer is checked for area conservation and disjointness, and one end-to-end case runs the entire warm-up flow inside `ctest`. Three external tools are used, all of them adversarially and none in the extraction path: a synthesis suite for SAT equivalence, an independent simulator to re-run every result, and a layout viewer.

c) Limitations: Cell identity was a lookup, because both designs kept their library names; stripped names would require cells classified by geometry, and a fully flattened layout would require transistor-level recognition. Well and substrate ties are inferred rather than extracted, since they are labelled on the well and the substrate, which no metal trace reaches; the tool reports how many it inferred. The input search is exhaustive, which is adequate at this size. Timing is ignored entirely, because extraction is topological.

IX. CONCLUSIONS

Three observations, in the order they mattered.

Read the file before writing code. Thirty seconds decided whether this was a geometry problem or a device-recognition problem, and the two differ by an order of magnitude in effort.

Build the verification before the result. A faulty extractor is indistinguishable from a correct one by inspection, and the warm-up exists precisely so that it does not have to be distinguished that way. Everything downstream — 312 of 312 samples, a unique solution, a message that names its own rule — was believable only because the machinery producing it had already been proven correct somewhere the answer was known.

Stop reading when reading becomes expensive. Eleven irregular regions appeared in no equation I could parse; they came out of 121 experiments that ran in a second. A netlist is a poor thing to read and a very good thing to experiment on.

APPENDIX A

THREE THINGS HIDDEN IN THE LAYOUT

None of these are needed to solve the puzzle. Each was found by a step that was already part of the work, which is why each is worth recording: they show what a routine question turns up when it is asked of the whole file rather than of the part one cares about.

A. A message below the die

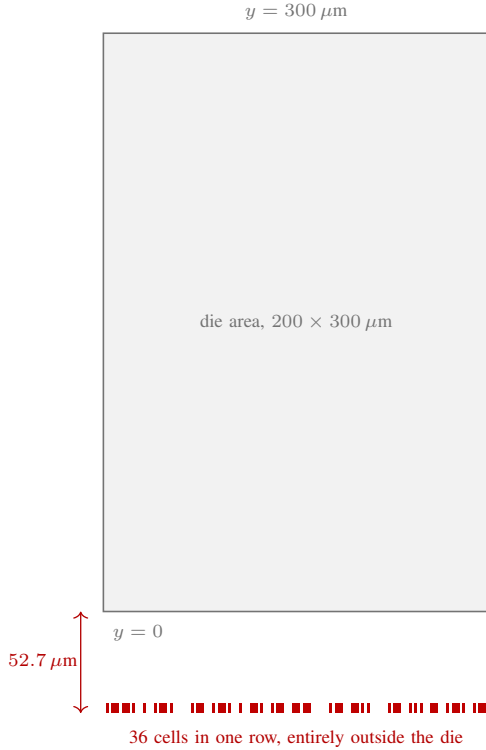
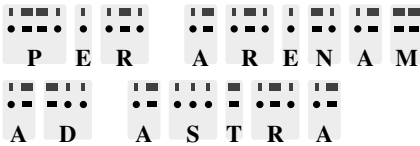


Fig. 5: Where the row actually is. The grey rectangle is the die area, drawn to scale from the layout’s own boundary layer; the red bars are the 36 placeholder cells, also to scale. They sit $52.7\ \mu\text{m}$ below the bottom edge, in a strip of the file that contains nothing else. No manufacturable geometry exists there, which is why they can be ignored by extraction and why they are easy to miss entirely.



A $1.38\ \mu\text{m}$ cell is a dot, a $4.14\ \mu\text{m}$ cell is a dash, and the three gap widths $1.38/4.14/9.66\ \mu\text{m}$ separate symbols, letters and words. Letters are spaced evenly here for legibility; their true positions are in Fig. 5.

Fig. 6: The decoding. Each cell of the row is drawn above the symbol it stands for and the letter it belongs to, and the tint marks the letter boundaries the gap widths imply. It reads PER ARENAM AD ASTRA — through the sand, to the stars.

a) How it surfaced: The first command I ran, before writing any extraction code, lists the cells the layout places. Two of them are not from any standard library. Every other

name in that listing is a documented standard cell, so I looked at what is inside these two and where they are placed.

TABLE II: The two cells in the layout that belong to no standard library. Neither is manufacturable geometry and neither carries a pin, so extraction ignores both.

Cell	Placed	Width	Contents
INTERNAL_3	21×	$1.38\ \mu\text{m}$	one rectangle, layer 200/0
INTERNAL_7	15×	$4.14\ \mu\text{m}$	one rectangle, layer 200/0

Three things stand out. Layer 200/0 is not defined by the process, so the shapes are not manufacturable geometry. Neither cell contains a pin, so extraction ignores both completely: they are electrically inert. And all 36 placements sit at $y = -52.7\ \mu\text{m}$, whereas the die area runs from $y = 0$ to $y = 300\ \mu\text{m}$. The entire row therefore lies *outside* the chip (Figure 5), in a strip of the file that contains nothing else.

b) What it is: Two widths in a ratio of one to three, and gaps that take exactly three distinct values, 1.38 , 4.14 and $9.66\ \mu\text{m}$. Two symbol lengths and three separator lengths is Morse code. Reading short as a dot, long as a dash, and the three gaps as intra-character, inter-character and inter-word separators gives Figure 6.

B. Four output strings where one would do

a) How it surfaced: Once `success` went high I read the output bus to get the answer, and got `(* TWO STARS *)`. Out of habit I then ran the same trace on the inputs I had been testing with all along — the all-zero grid used to characterise the counters, and the all-ones grid used to check the saturation behaviour — expecting the same failure string from both. They produced different strings (Table III).

TABLE III: The four strings the output stage can emit, and the case each covers. No fifth message is reachable.

Input	O spells	Case
the solution	<code>(* TWO STARS *)</code>	names its own rule
empty grid	<code>EMPTY SKY</code>	nothing marked
all cells marked	<code>BIG BANG</code>	everything marked
anything else	<code>TRY AGAIN</code>	ordinary failure

b) Confirming there is no fifth: The output stage is 23 flip-flops, of which 12 form the character generator. Rather than reverse its feedback sequence I enumerated all 2^{12} states of that generator against each of the four end states of the checker, and collected every byte it can emit. In the three failure modes the reachable alphabet is exactly the set of letters in the string concerned — AGINRTY for TRY AGAIN, and so on — which is a small confirmation that those strings are stored rather than computed. No fifth message is reachable.

C. A timestamp that cannot exist

a) How it surfaced: To replay the vendor’s sample waveform through the extracted netlist I had to parse it, which meant reading its header. The header is six lines (Listing 2), and two of them are jokes.

```
$date
  Sat Dec 31 23:59:60 2016
$end
$version
  Leave no stone unturned! But for this
  file, consider looking at it in a
  waveform viewer instead.
$end
```

Listing 2: The header of the supplied waveform. Both the date and the version string are jokes.

b) What it is: A minute has sixty seconds, numbered 00 to 59, so second 60 should not exist. It did once: a leap second was inserted at the end of 2016, making 23:59:60 on 31

December a real instant that lasted exactly one second. It is the only timestamp of its kind that year.

The version string is the broader hint. In hindsight it points at the row of Morse cells, which is the one thing in the file that lies outside the die and is easy to leave unturned.

APPENDIX B

THE RECOVERED DESIGN IN FULL

Figure 8 is the block diagram referred to in Section IV, rotated for legibility. Figure 7 gives the next-state functions, with $step = enable \wedge \neg done$ and $R(r, c)$ the region map of Figure 3a. Table IV accounts for every flip-flop in the design.

$$col' = step ? (col=10 ? 0 : col+1) : col \quad (1)$$

$$row' = (step \wedge col=10) ? (row=10 ? 0 : row+1) : row \quad (2)$$

$$done' = done \vee (step \wedge col=10 \wedge row=10) \quad (3)$$

$$hist' = step ? \{hist[11:1], I\} : hist \quad (4)$$

$$touching = (col \neq 0) h_1 \vee (col \neq 10) h_{10} \vee h_{11} \vee (col \neq 0) h_{12} \quad (5)$$

$$col_cnt[col] += I \cdot step, \quad reg_cnt[R(row, col)] += I \cdot step, \quad total += I \cdot step \quad (6)$$

$$row_cnt' = (step \wedge col=10) ? 0 : row_cnt + I \cdot step \quad (7)$$

$$viol_adj' = viol_adj \vee (I \wedge step \wedge touching) \quad (8)$$

$$viol_row' = viol_row \vee (step \wedge col=10 \wedge row_cnt + I \neq 2) \quad (9)$$

$$success' = (done \wedge \neg done_d) ? (\neg viol_adj \wedge \neg viol_row \wedge \bigwedge_c col_cnt[c]=2 \wedge \bigwedge_R reg_cnt[R]=2 \wedge total=22) : success \quad (10)$$

Fig. 7: The recovered next-state functions. Every line was read out of the extracted netlist by symbolic expansion, with the single exception of the region map R , which was measured by perturbation.

TABLE IV: All 92 flip-flops, grouped by the role their equations reveal. Bit weights come from the carry structure rather than from assumption.

Register	Flip-flops	Width	Block	Role
col, row	$q_{37..40}, q_{65..68}$	4+4	A	scan position, modulo eleven in each axis
done, done_d	q_{36}, q_0	1+1	A	the finished flag, and the same flag delayed one cycle
hist	$q_{41..53}$	12	B	shift register of $\mathbb{I}$; its taps give the neighbours
col_cnt	22 flip-flops	11×2	C	marks per column, saturating
reg_cnt	22 flip-flops	11×2	C	marks per region, saturating
row_cnt	q_{54}, q_{56}	2	C	marks in the current row, cleared at each row end
total	$q_{57..64}$	8	C	marks in the whole grid
viol_adj, viol_row	q_{46}, q_{55}	1+1	D	sticky vetoes
success	q_2	1	D	the output
message sequencer	23 flip-flops	—	E	drives $\mathcal{O}[7:0]$

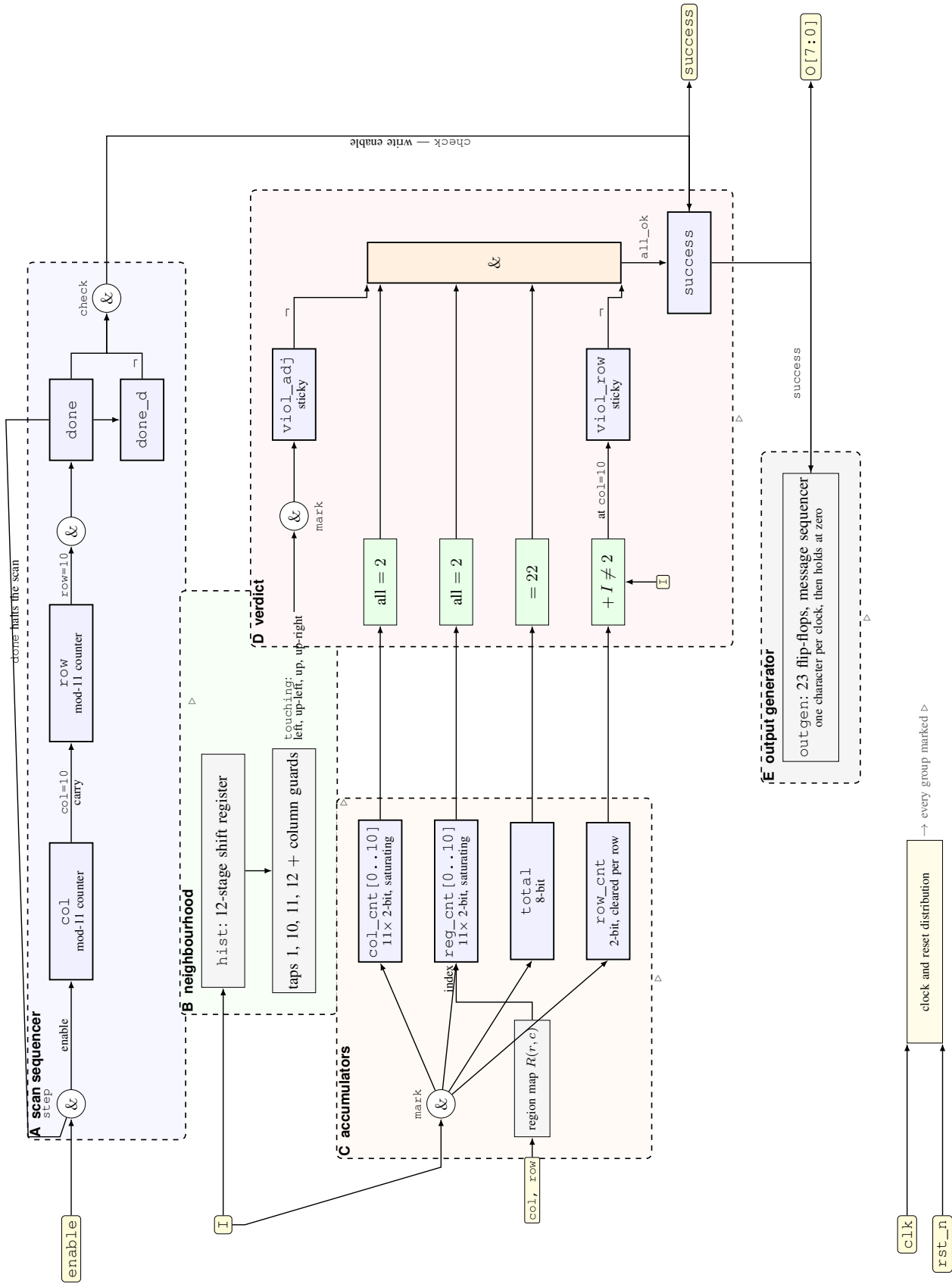


Fig. 8: The recovered chip, with all five ports. Block **A** scans the grid at one cell per clock and latches `done` on the last one, making `check` true for exactly one cycle. Block **B** delays the input by twelve cycles, so that taps 1, 10, 11 and 12 are the four already-scanned neighbours. Block **C** accumulates the mark count per column, per region, per row and overall. Block **D** compares all four against constants, vetoes on either sticky flag, and writes the result once. Block **E** turns the verdict into an ASCII string. Everything here was read out of the next-state equations except the region map, which was measured.